

OrquestraPOD

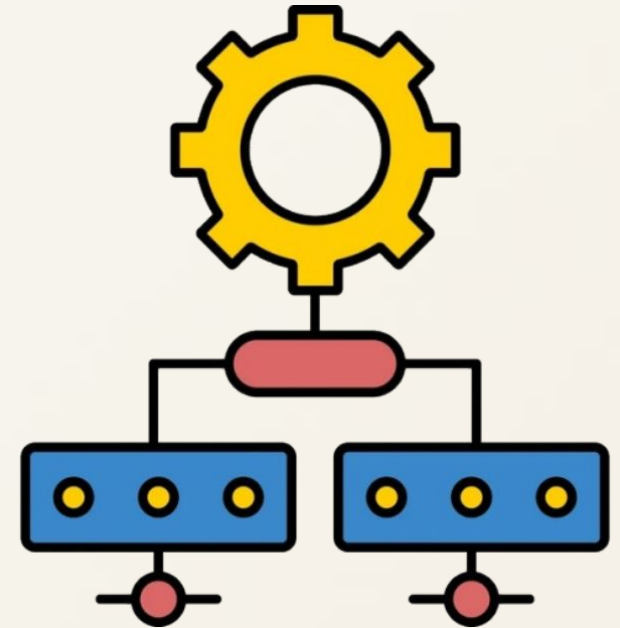
Orquestrador de PODs com escalonamento

Trabalho I — Sistemas Operacionais: Análise e Aplicações

Unisinos · 2026/1

Aluno: Eduardo Rodrigues Graf

Data: 04/06/2026



O problema (analogia do restaurante)

Imagine um restaurante movimentado:

📋 Chegam **pedidos** o tempo todo → cada pedido é uma **tarefa a executar**.

👤 Um **gerente** recebe os pedidos e decide **para qual cozinha** enviar cada um.

👨‍🍳 Várias **cozinhas**, cada uma com **capacidade diferente** (mais fogões, mais perto do salão).

Mandar tudo para a mesma cozinha = sobrecarga.

O gerente precisa **distribuir de forma inteligente**, olhando o que cada cozinha tem livre.

Kubernetes faz isso, mas para programas.



Os conceitos (traduzindo a analogia)

NO RESTAURANTE	NO KUBERNETES	O QUE É
Pedido	POD	a menor unidade que roda — "uma aplicação empacotada"
Gerente	Master	o cérebro: decide onde cada POD roda
Cozinha	Worker	a máquina que executa os PODs (CPU, memória, disco)
Decisão	Escalonador	o algoritmo que escolhe o Worker para cada POD



Worker = um computador com capacidade limitada; vai enchendo conforme recebe PODs.



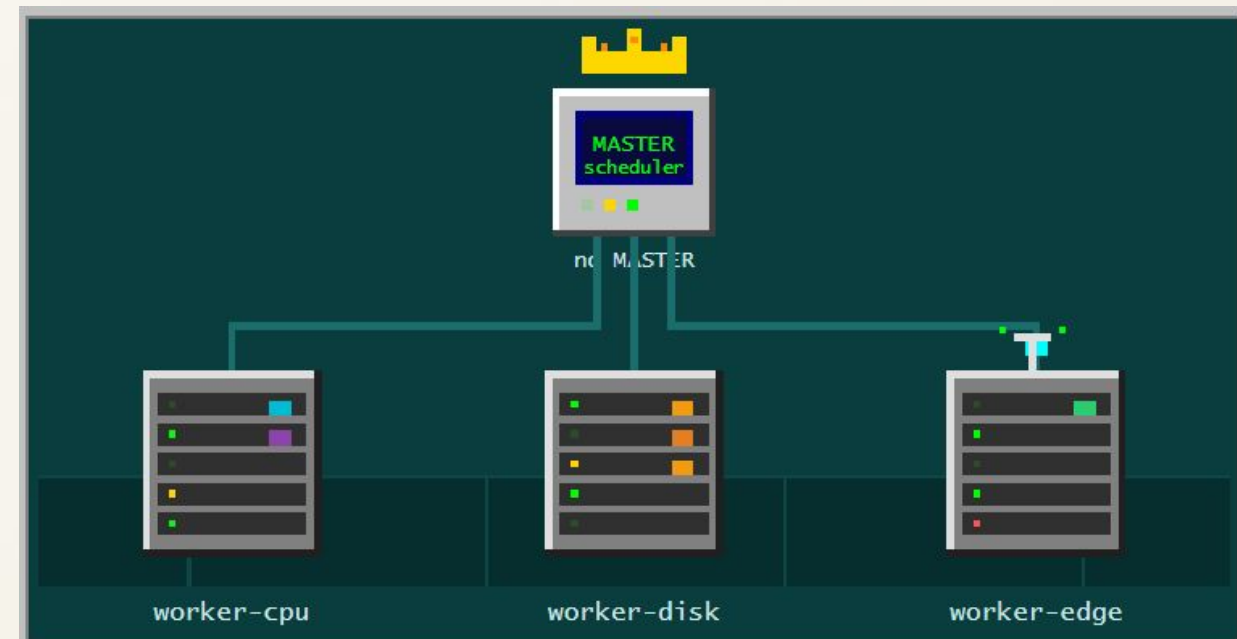
Cluster = Master + Workers trabalhando juntos. Neste projeto: **1 Master + 3 Workers**.

A proposta do trabalho

O escalonador padrão do Kubernetes é simples de propósito:
ao decidir onde colocar um POD, ele olha **apenas 2 métricas** → CPU e memória.

A missão (o enunciado):

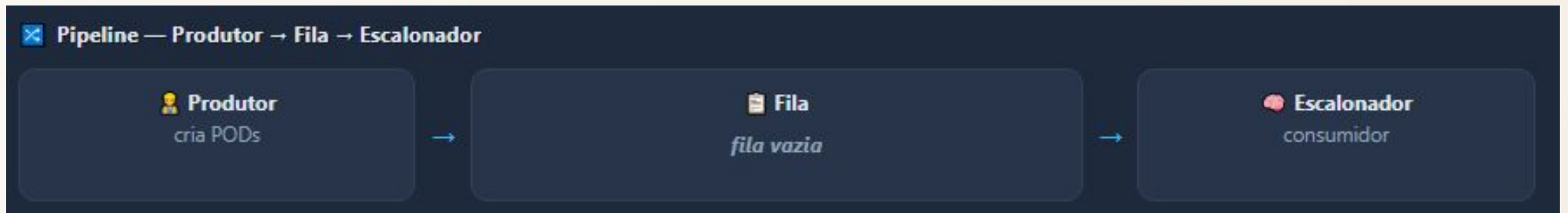
- ✓ Montar o cluster (Master + ≥ 2 Workers).
- ✓ Criar um escalonador com ≥ 3 métricas.
- ✓ Gerar e distribuir + **de uma dezena de PODs**.
- ✓ **Estatísticas + comparação** com o padrão.



A aplicação

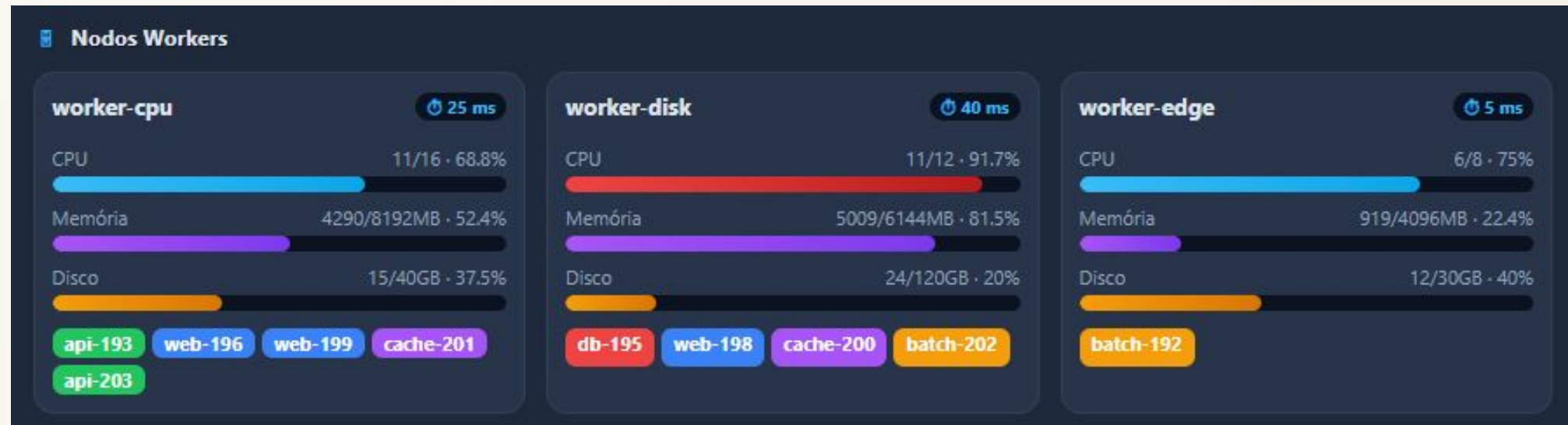
O sistema executa o fluxo:

1. Um **produtor** cria PODs continuamente (tipos: web, api, cache, batch, db).
2. Os PODs entram numa **fila** de processamento.
3. O **escalonador** tira da fila e escolhe o melhor Worker.
4. O POD **ocupa recursos** (barras sobem); ao fim, **libera** (descem).



Demonstração ao vivo: o painel

- Os **KPIs** no topo: PODs criados, na fila, executando e concluídos.
- O **pipeline** Produtor → Fila → Escalonador e o comportamento da fila.
- Os **cards dos Workers**: barras de CPU / Memória / Disco flutuando dinamicamente.
- O **log de eventos** atualizando a cada segundo em tempo real.



As 4 métricas do escalonador

#	MÉTRICA	PADRÃO K8S?	OBSERVA O QUÊ
1	CPU	✓ já olha	núcleos livres no nó
2	Memória	✓ já olha	RAM livre disponível
3	Disco	✗ a mais	espaço livre em disco
4	Latência de rede	✗ a mais	quão rápido/perto é o Worker

Workers feitos diferentes de propósito:

worker-cpu → muita CPU/memória

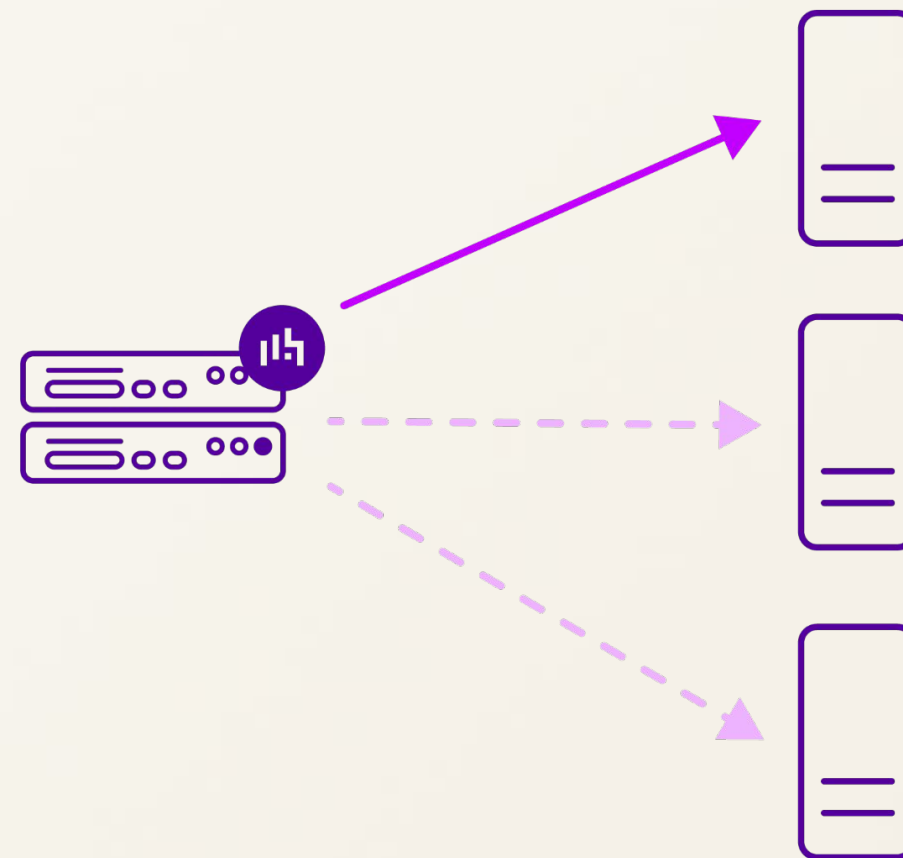
worker-disk → muito disco (atrai batch/db)

worker-edge → latência baixíssima (atrai web/cache)

Decisão pelas métricas

A decisão acontecendo:

-  Um POD **batch** ou **db** (precisa de muito disco) → o escalonador direciona para o **worker-disk**.
-  Um POD **web** ou **cache** (sensível a latência) → cai no **worker-edge** (5 ms).
-  Passando o mouse sobre um POD no painel, veem-se seus requisitos exatos.



Como o escalonador decide

Para cada POD, o algoritmo executa duas etapas:

▼ **1. Filtrar** — descarta Workers onde o POD não cabe (falta CPU, RAM ou Disco).

★ **2. Pontuar e escolher** — dá uma nota a cada candidato aprovado, favorecendo o **equilíbrio** (nó mais folgado) e a **latência** (nó mais rápido).

→ **Maior nota vence.** Se nenhum couber, o POD vira "Não alocável".

 scheduler.py

```
1 def escolher(self, workers, pod):  
2     # 1) filtra  
3     candidatos = [w for w in workers if w.comporta(pod)]  
4     if not candidatos:  
5         return None  
6     # 2) pontua  
7     return max(candidatos, key=lambda w: self.pontuar(w, pod))
```

O lado de Sistemas Operacionais

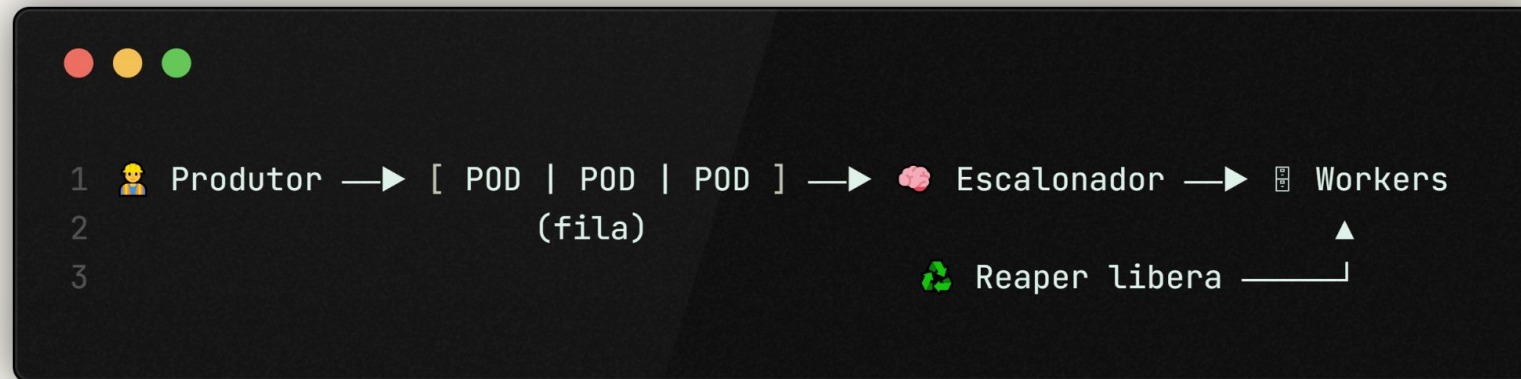
Padrão produtor/consumidor com 3 threads em paralelo:

 **Produtor** → cria PODs e coloca na **fila** thread-safe.

 **Consumidor (escalador)** → tira da fila e aloca no Worker ideal.

 **Reaper** → remove PODs que cumpriram o tempo, liberando recursos.

Uma fila no meio segura a produção quando o consumidor está ocupado, e um **lock** (exclusão mútua) protege o estado compartilhado.



Comparação: proposto × padrão

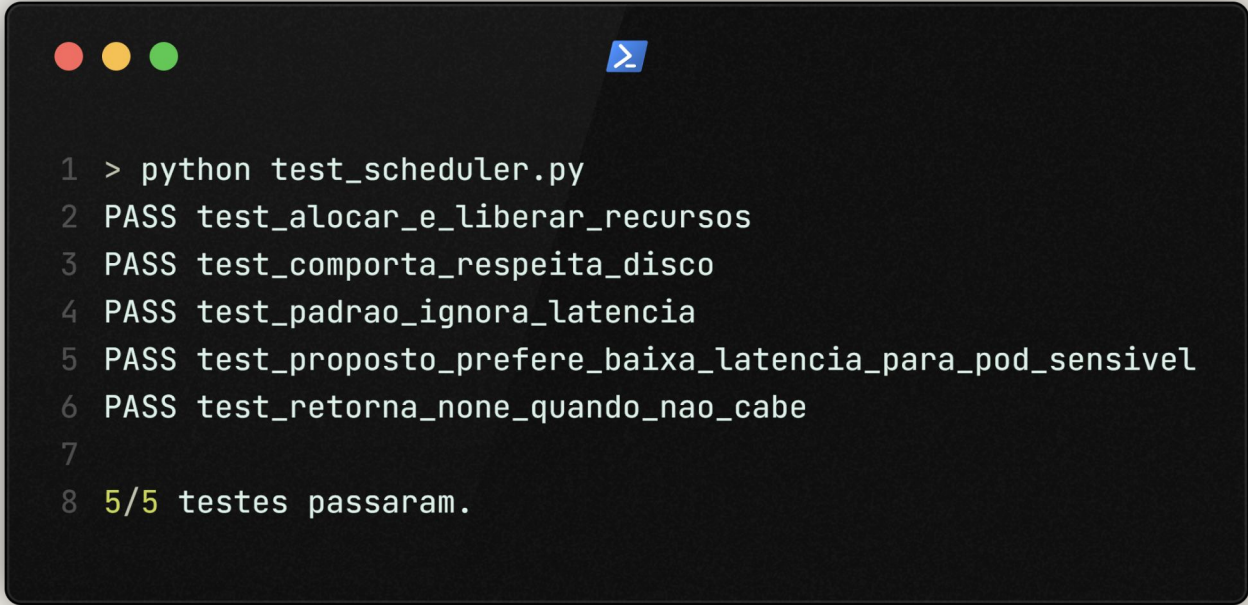
Mesma carga de PODs, dois escalonadores, Workers idênticos.

CRITÉRIO	PROPOSTO (4 MÉTRICAS)	PADRÃO K8S (2 MÉTRICAS)
PODs alocados	mais (enxerga disco)	menos
Latência média	menor (usa worker-edge)	maior
Balanceamento	equilibrado	equilibrado

O proposto, por enxergar **disco**, encaixa mais PODs evitando gargalos invisíveis. Por olhar **latência**, baixa a latência média ao direcionar inteligentemente ao nó rápido. O padrão, cego para essas métricas, perde nos dois pontos.

Testes

A lógica central contém testes automatizados.

A terminal window with a dark background and a blue prompt character. It shows the execution of a Python script and its output. The output lists five tests, all of which passed, followed by a summary line.

```
1 > python test_scheduler.py
2 PASS test_alocar_e_liberar_recursos
3 PASS test_comporta_respeita_disco
4 PASS test_padrao_ignora_latencia
5 PASS test_proposto_preferencia_baixa_latencia_para_pod_sensivel
6 PASS test_retorna_none_quando_nao_cabe
7
8 5/5 testes passaram.
```

Eles provam que o escalonador:

- ✓ **respeita o disco** (nunca aloca onde não cabe);
- ✓ **prefere baixa latência** para os PODs sensíveis à rede.

Conclusão

O que foi entregue:

- ✓ Cluster Master + 3 Workers simulado fielmente em Python.
- ✓ Escalonador **multi-métrica** (CPU, memória, disco, latência).
- ✓ Visualização em painel web, estatísticas e comparação com o padrão K8s.
- ✓ Aplicação real de conceitos de SO (threads, exclusão mútua, fila).